

Corso di Laurea in Ingegneria e Scienze Informatiche

Fancy Title

Tesi di laurea in:
PROGRAMMAZIONE AD OGGETTI

Relatore

Mirko Viroli

Candidato

Alessandro Gardini

Correlatore

Gianluca Aguzzi

Sommario

Max 2000 characters, strict.

Optional. Max a few lines.

Indice

Sommario	iii
1 Introduzione	1
2 Background	3
2.1 Introduzione all'Aggregate Computing	3
2.2 Progetto allo stato attuale	4
2.2.1 Obiettivo	5
2.2.2 Architettura e funzionamento	6
2.2.3 Vincoli	8
3 Analisi	11
3.1 Obiettivi	11
3.2 Vincoli	12
3.3 Requisiti	12
3.3.1 Requisiti funzionali	13
3.3.2 Requisiti non funzionali	13
3.3.3 Profili di utilizzo	13
3.4 Modellazione delle Entità	14
4 Progettazione	17
4.1 Architetturale rispetto al sistema	17
4.2 Architetturale di dettaglio dell'applicativo	20
4.2.1 Progettazione delle connessioni	23
5 Implementazione	31
5.1 Stack tecnologico e organizzazione del codice	31
5.2 Principali decisioni implementative	33
5.2.1 Implementazione delle Connessioni	36
6 Valutazione	43
6.1 Testing automatico	43

INDICE

7	Conclusione e Sviluppi Futuri	47
		49
	Bibliografia	49

Elenco delle figure

2.1	Connessioni tra i moduli del sistema Emerge nella versione iniziale. Le frecce indicano il flusso dei messaggi MQTT (PUB/SUB) e le chiamate HTTP verso il neighborhood-system	7
3.1	Schema delle entità di dominio.	15
4.1	Architettura del sistema con l'introduzione del gateway. I componenti evidenziati in blu rappresentano le aggiunte introdotte.	19
4.2	Architettura interna del gateway: connessioni, bus e cache.	22
4.3	Diagramma di sequenza del meccanismo a callback. Gli aggiornamenti non sono sincroni. Gli aggiornamenti lato client non sono riportati, poiché equivarrebbero a una semplice chiamata Hypertext Transfer Protocol (HTTP).	26
4.4	Diagramma di sequenza del meccanismo a Server-Sent Events. Come per il caso precedente, gli aggiornamenti non sono sincroni e quelli lato client non sono riportati.	27
4.5	Diagramma di sequenza del meccanismo a WebSocket. Come per i casi precedenti, gli aggiornamenti non sono sincroni. I tre blocchi non sono necessariamente sequenziali: lo schema fornisce una rappresentazione puramente schematica dell'interazione.	29

Capitolo 1

Introduzione

Capitolo 2

Background

Questo capitolo fornisce gli elementi di contesto necessari a comprendere il lavoro di tesi. Si introduce dapprima il paradigma dell'aggregate computing, presentandone i fondamenti teorici e i framework su cui si basa (sezione 2.1); si descrive quindi la sua applicazione concreta nel sistema *Emerge*, il dimostratore realizzato dal gruppo di ricerca e punto di partenza del lavoro qui descritto, illustrandone l'obiettivo, l'architettura e i vincoli strutturali che ne sono emersi (sezione 2.2). Questi vincoli costituiscono il punto di partenza per l'analisi condotta nel capitolo successivo.

2.1 Introduzione all'Aggregate Computing

L'aggregate computing è un paradigma di programmazione che affronta la complessità dello sviluppo di sistemi distribuiti e decentralizzati astruendo la computazione su una rete di dispositivi interagenti come se fosse un'unica entità collettiva. L'elemento concettuale alla base del paradigma è il campo computazionale, una funzione che associa a ogni punto dello spazio e del tempo occupato dai dispositivi un valore, manipolabile attraverso composizione funzionale. Operativamente, ogni dispositivo esegue periodicamente, in modo asincrono rispetto agli altri, un round computazionale: percepisce lo stato dei propri vicini e dell'ambiente, esegue localmente lo stesso programma aggregato e ne comunica l'esito, contribuendo così, attraverso l'iterazione di questo ciclo nel tempo, all'emergere del comportamento collettivo desiderato a partire da sole interazioni locali. Questa astrazione con-

sente di specificare in modo dichiarativo il comportamento desiderato dell'intero sistema, con meccanismi di coordinamento intrinsecamente robusti e scalabili al variare del numero di dispositivi partecipanti.

Dal punto di vista formale, queste astrazioni trovano fondamento nel field calculus, un calcolo che fornisce i costrutti di base – diffusione dell'informazione, raccolta e aggregazione dei dati, evoluzione nel tempo – a partire dai quali è possibile costruire funzioni di livello più alto. Su queste basi sono state sviluppate librerie di comportamenti collettivi auto-stabilizzanti, sia di natura generica sia specializzati per domini applicativi specifici, tra cui la robotica di sciame e il coordinamento gerarchico: è proprio in questo filone che si inserisce *MacroSwarm* [ACV24], la libreria su cui si basa il progetto descritto in questa tesi.

Negli anni sono stati sviluppati diversi framework a supporto dell'aggregate computing, ciascuno dei quali implementa i costrutti fondamentali del field calculus offrendo al contempo un proprio approccio all'esecuzione e all'integrazione con l'ambiente circostante. Tra questi, *ScaFi* [CVAP22] si distingue come ambiente di sviluppo basato su Scala, pensato per essere utilizzabile sia in simulazione sia in scenari di esecuzione reale, e offre un'API di alto livello per la definizione concisa di computazioni aggregate, mantenendo al contempo l'estensibilità necessaria per l'integrazione con diversi ambienti di esecuzione. Un altro framework rilevante è *FCPP* [AT24], le cui implementazioni leggere ne consentono l'esecuzione anche su microcontrollori e schede embedded a basso consumo, rendendo il paradigma applicabile anche a reti IoT e a sistemi robotici con risorse computazionali limitate, sebbene nel progetto descritto in questa tesi la computazione aggregata sia demandata a un nodo esterno anziché eseguita direttamente a bordo dei singoli robot, come illustrato nella sezione seguente.

2.2 Progetto allo stato attuale

La sezione precedente ha introdotto i fondamenti teorici dell'aggregate computing, illustrando il paradigma e i framework – *ScaFi* e *FCPP* – su cui esso si basa. Questa sezione ne presenta l'applicazione concreta nel sistema *Emerge*, il dimostratore realizzato dal gruppo di ricerca e punto di partenza del lavoro di tirocinio descritto in questa tesi: si illustrano dapprima l'obiettivo che ne ha guidato lo

sviluppo (sezione 2.2.1), quindi l'architettura attraverso cui tale obiettivo è stato realizzato (sezione 2.2.2); infine si analizzano i vincoli strutturali emersi da queste scelte progettuali (sezione 2.2.3), che costituiscono il punto di partenza per l'analisi presentata nel capitolo successivo.

2.2.1 Obiettivo

Il progetto di partenza, documentato in [ABB⁺25], nasce dall'esigenza di colmare il divario tra le fondamenta teoriche dell'aggregate computing e la sua applicazione pratica su sistemi robotici reali. Nonostante il paradigma vanti oltre un decennio di ricerca e diversi framework consolidati, le dimostrazioni su hardware fisico rimanevano limitate, rendendo difficile la transizione dal livello di maturità tecnologica TRL 3 (validazione in laboratorio) al TRL 6 (prototipo funzionante in ambiente reale) [OEJ15].

L'obiettivo principale era quindi realizzare un **dimostratore pratico e open-source** per il coordinamento di squadre di robot mobili, basato sui principi dell'aggregate computing. Il dimostratore è costruito sulla libreria *MacroSwarm* [ACV24] e sul framework *ScaFi*, ed è stato concepito per essere flessibile, modulare e riutilizzabile da chiunque voglia replicare o estendere scenari analoghi.

Il progetto perseguiva tre obiettivi dimostrativi specifici. Il primo (**G1**) riguardava la formazione spaziale: mostrare come un gruppo di robot riesca a disporsi autonomamente in pattern geometrici – come cerchio o linea – attraverso la sola computazione aggregata, senza alcun controllo centralizzato esplicito. Il secondo (**G2**) riguardava la capacità di autoripristino (*self-healing*): verificare che il sistema fosse in grado di ripristinare autonomamente la formazione desiderata in seguito a perturbazioni, come lo spostamento fisico di un robot o la rimozione temporanea di un nodo dalla squadra. Il terzo (**G3**) riguardava l'*indipendenza dalla topologia*: dimostrare che la logica di coordinamento rimanesse robusta e coerente al variare del numero di robot attivi e della configurazione della rete.

Oltre agli aspetti tecnici, il progetto aveva anche una finalità divulgativa: rendere accessibili e comprensibili i concetti dell'aggregate computing a un pubblico eterogeneo, inclusi visitatori non tecnici e bambini, attraverso comportamenti visivamente intuitivi. La validazione è avvenuta in occasione della Notte Europea

dei Ricercatori, dove il dimostratore è stato presentato in un contesto pubblico e interattivo.

2.2.2 Architettura e funzionamento

Il sistema realizzato, chiamato *Emerge*, è composto da quattro moduli principali che realizzano concretamente le astrazioni architetturali proposte nel modello di riferimento: `EnvironmentProvider`, `Orchestrator` ed `EnvironmentUpdate`. Ciascun modulo ha un ruolo ben definito e comunica con gli altri esclusivamente attraverso un broker Message Queuing Telemetry Transport (MQTT) centralizzato (Mosquitto), che funge da unico bus di comunicazione dell'intero sistema. Questa scelta architetturale garantisce un disaccoppiamento naturale tra i componenti. Le connessioni tra i moduli sono illustrate in fig. 2.1.

Il modulo `aruco-detector`, realizzato in Python con le librerie OpenCV e paho-mqtt, svolge il ruolo di `EnvironmentProvider`. Acquisisce in continuo il flusso video da una telecamera, individua i marker ArUco – piccoli tag visivi applicati fisicamente su ciascun robot – e ne ricava posizione e orientamento, pubblicando i dati aggiornati sul broker MQTT.

Il modulo `neighborhood-system`, sviluppato anch'esso in Python tramite Flask e paho-mqtt, contribuisce alla costruzione dell'ambiente computazionale ricalcolando dinamicamente il vicinato di ciascun robot a ogni aggiornamento di posizione ricevuto dal broker. Il calcolo supporta due modalità configurabili a runtime: `full`, in cui tutti i robot attivi sono considerati vicini l'uno dell'altro, e `radius`, in cui vengono considerati vicini solo i robot entro un raggio specificato. Il vicinato aggiornato viene a sua volta pubblicato sul broker, e per consentirne la riconfigurazione senza riavviare il sistema il modulo espone alcuni endpoint HTTP, utilizzati sia dalla Dashboard sia a scopo di debug.

Il modulo `aggregate-runtime`, scritto in Scala, costituisce il nucleo computativo del sistema e svolge il ruolo di `Orchestrator` ed `EnvironmentUpdate`. È organizzato in due sottomoduli: un sottomodulo `core`, che definisce in astratto la pipeline di controllo `EnvironmentProvider` $\rightarrow$ `Orchestrator` $\rightarrow$ `EnvironmentUpdate`, parametrizzata sui tipi dominio-specifici del sistema in modo da poter essere riutilizzata per qualsiasi squadra di robot indipendentemente da come questi sono iden-

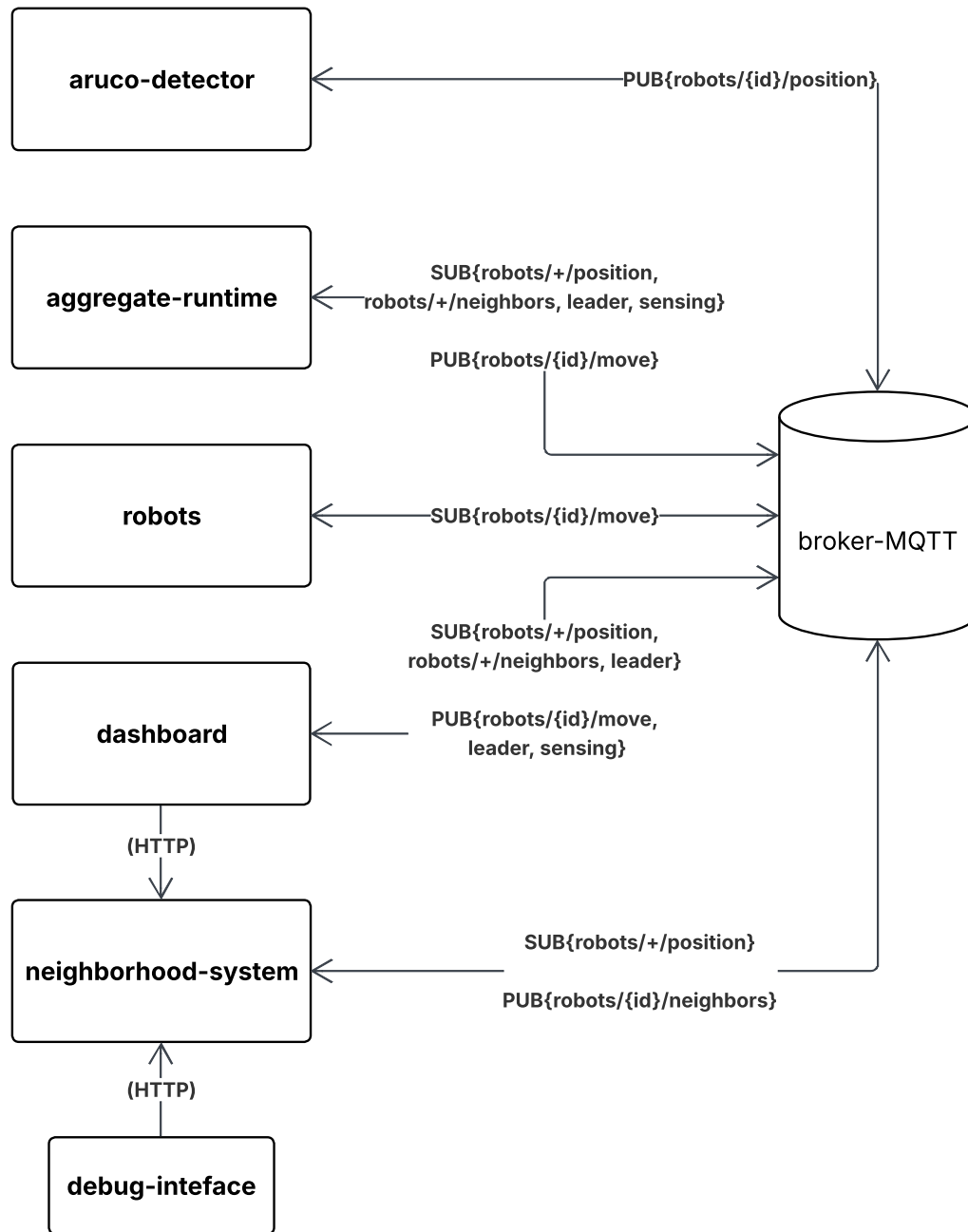


Figura 2.1: Connessioni tra i moduli del sistema Emerge nella versione iniziale. Le frecce indicano il flusso dei messaggi MQTT (PUB/SUB) e le chiamate HTTP verso il **neighborhood-system**.

tificati, localizzati e attuati; e un sottomodulo **demo**, che ne fornisce una realizzazione concreta per lo sciame fisico, collegando le informazioni ricevute dal broker all'esecuzione di un round ScaFi per ciascun robot e traducendone l'esito in un comando motore. I comportamenti collettivi dello sciame – tra cui **PointTheLeader**, **VFormation** e **CircleFormation** – sono definiti come scenari ScaFi selezionabili a runtime, senza necessità di riavviare il sistema.

Infine, il modulo **dashboard**, sviluppato in React e TypeScript, fornisce all'operatore un'interfaccia di visualizzazione e controllo in tempo reale: riceve dal broker lo stato aggiornato dello sciame e può a sua volta inviare comandi, oltre a comunicare direttamente con il **neighborhood-system** tramite HTTP per modificarne la configurazione senza passare dal broker.

2.2.3 Vincoli

L'architettura di *Emerge*, pur risultando efficace per gli obiettivi dimostrativi per cui è stata concepita, presenta alcuni vincoli strutturali che emergono direttamente dalle scelte implementative adottate.

Il vincolo più pervasivo riguarda il broker MQTT, scelto in fase di sviluppo per la sua semplicità come unico mezzo di comunicazione dell'intero sistema. Questa decisione ha conseguenze su tutti i livelli: non solo i moduli software si parlano esclusivamente tramite Mosquitto, ma anche i robot fisici devono obbligatoriamente supportare MQTT per poter partecipare allo sciame. Ne deriva che qualsiasi robot privo di un client MQTT nativo – o che utilizzi un protocollo di comunicazione diverso – risulta di fatto incompatibile con il sistema senza interventi di adattamento. La varietà di protocolli oggi disponibili in ambito IoT rende questo un limite non trascurabile in ottica di estensibilità.

Un caso particolare è rappresentato dalla **dashboard**: poiché i browser non consentono connessioni TCP dirette verso broker MQTT arbitrari, il modulo ricorre a **MQTT over WebSocket** (porta 9001), che incapsula i messaggi MQTT all'interno di una connessione WebSocket. Il meccanismo funziona, ma introduce un livello di wrapping aggiuntivo che conferma come anche il componente di interfaccia sia vincolato al paradigma MQTT, seppur con un trasporto differente.

Come nota finale, la localizzazione dei robot dipende interamente dalla presenza di marker ArUco fisici e dalla visibilità della telecamera: un robot non dotato di tali marker non può essere percepito dal sistema, indipendentemente da come comunica.

Capitolo 3

Analisi

Il capitolo precedente ha messo in luce i limiti strutturali del sistema *Emerge*, in particolare la sua rigida dipendenza dal protocollo MQTT. In questo capitolo tali limiti vengono tradotti in un problema di progettazione: si definiscono dapprima l'obiettivo generale del lavoro (sezione 3.1) e i vincoli entro cui esso deve essere perseguito (sezione 3.2); se ne derivano poi i requisiti funzionali e non funzionali (sezione 3.3); infine si presenta il modello di dominio che fungerà da riferimento per le scelte progettuali del capitolo successivo (sezione 3.4).

3.1 Obiettivi

L'obiettivo generale è stato quindi rendere il sistema **protocol-agnostic**: ridurre, cioè, il grado di accoppiamento rispetto a un singolo protocollo, aprendo la possibilità di integrare robot e client che utilizzino protocolli di comunicazione eterogenei senza dover modificare il nucleo del sistema.

Un aspetto centrale emerso durante lo studio del problema è stata la distinzione tra due mondi distinti: quello interno al sistema, composto dai moduli già esistenti e dai robot che comunicano tramite MQTT, e quello esterno, composto da client non ancora integrati o gestiti in modo non ottimale – come la dashboard, che ricorre a MQTT over WebSocket per aggirare i limiti del browser – e da robot futuri che potrebbero non supportare MQTT. Da qui, l'obiettivo già enunciato si è declinato nel rendere il protocollo interno indipendente da quelli usati per comunicare con

l'esterno, in modo che il sistema potesse supportare client e dispositivi futuri senza che questi fossero vincolati a MQTT.

3.2 Vincoli

Il contesto in cui si è operato imponeva due vincoli principali. Il primo riguardava la non modificabilità dei moduli già sviluppati: qualsiasi intervento che avesse richiesto di riscrivere o alterare il comportamento dei componenti esistenti era da considerarsi estraneo agli obiettivi del progetto. Questo vincolo stabiliva quindi, già in questa fase, che il contributo doveva integrarsi nel progetto esistente per estensione, non per sostituzione.

Il secondo vincolo, strettamente connesso al primo, riguardava il broker MQTT interno. Sebbene la dipendenza da MQTT rappresenti proprio il limite che si intendeva superare, la sua rimozione avrebbe comportato una riscrittura completa di tutti i moduli del sistema. MQTT rimaneva quindi il protocollo interno, e qualsiasi soluzione doveva operare al di sopra di questa viscosità tecnologica¹ piuttosto che tentare di eliminarla.

3.3 Requisiti

Il capitolo precedente ha messo in luce i limiti strutturali del sistema *Emerge*, in particolare la sua rigida dipendenza dal protocollo MQTT. In questo capitolo tali limiti vengono tradotti in un problema di progettazione: si definiscono dapprima l'obiettivo generale del lavoro (sezione 3.1) e i vincoli entro cui esso deve essere perseguito (sezione 3.2); se ne derivano poi i requisiti funzionali e non funzionali (sezione 3.3); infine si presenta il modello di dominio che fungerà da riferimento per le scelte progettuali del capitolo successivo (sezione 3.4). I requisiti individuati si articolano su due livelli: funzionali, relativi alle capacità che il sistema deve offrire, e non funzionali, relativi alle qualità con cui tali capacità devono essere

¹Con *viscosità tecnologica* si intende la resistenza al cambiamento che emerge quando una tecnologia è così profondamente integrata in un sistema da rendere la sua sostituzione sproporzionatamente costosa.

garantite. A questi si affianca un'analisi dei profili dei client che il sistema dovrà servire, utile a guidare le scelte progettuali successive.

3.3.1 Requisiti funzionali

Il sistema doveva esporre verso l'esterno un insieme di protocolli di comunicazione eterogenei, ciascuno con il proprio modello di interazione, permettendo a client diversi di accedere alle stesse risorse interne attraverso l'interfaccia più adatta alle proprie capacità. Per ciascun protocollo esposto, doveva essere possibile sia leggere il valore aggiornato di una risorsa sia aggiornarla, preservando verso l'esterno la semantica di notifica continua tipica del modello publish/subscribe, indipendentemente dal protocollo effettivamente usato. Tra le modalità di interazione supportate, il sistema doveva prevedere sia un modello request/response, per client che interrogano le risorse in modo puntuale, sia un modello a notifica, per client che necessitano di aggiornamenti continui senza dover interrogare ripetutamente il sistema.

Doveva inoltre essere possibile aggiungere nuovi protocolli in futuro per supportare client con capacità diverse.

3.3.2 Requisiti non funzionali

Data la natura IoT del sistema, il modulo doveva essere in grado di gestire un elevato numero di connessioni simultanee, mantenendo latenze contenute anche sotto carico. Aggiungere un nuovo protocollo non avrebbe dovuto richiedere modifiche al nucleo già esistente del modulo, ma solo l'implementazione di una nuova componente aggiuntiva. Infine, il sistema doveva essere progettato in modo da facilitare il testing automatico, sia per verificare il comportamento dei singoli componenti sia per garantire la correttezza dell'integrazione complessiva.

3.3.3 Profili di utilizzo

I client che dovranno essere serviti non sono omogenei né per capacità né per modello di interazione. È utile distinguere tre profili principali.

Il primo è quello del client browser: un'applicazione web come la dashboard attuale, che non può aprire connessioni MQTT native e necessita quindi di un'interfaccia basata su protocolli HTTP o derivati.

Il secondo è quello del dispositivo embedded: un robot o sensore con risorse computazionali e di rete limitate, per cui potrebbe non essere praticabile l'uso di stack TCP completi, e che beneficia di protocolli progettati per ambienti vincolati.

Il terzo è quello del client generico: un sistema esterno di supervisione o un operatore che interagisce con il sistema in modo puntuale, privilegiando la semplicità di integrazione tramite interfacce standard e ben documentate.

Questi tre profili non esauriscono tutti i casi possibili, ma guidano le scelte progettuali relative ai protocolli esposti dal gateway.

3.4 Modellazione delle Entità

Prima di procedere alla progettazione della soluzione, è stato necessario definire con precisione il modello del dominio: ogni risorsa che questo servizio avrebbe dovuto esporre verso l'esterno doveva corrispondere a un'entità ben identificata, con una semantica chiara e stabile. A tale scopo sono stati analizzati i topic MQTT già presenti nel progetto, esaminandone la struttura e il payload, e da questi è stato condotto un lavoro di astrazione e generalizzazione che ha portato a un modello capace di accogliere anche categorie di dati non ancora presenti, come quelle derivanti da sensori ambientali che potranno essere aggiunti in futuro a ciascun robot.

Il risultato di questa fase è sintetizzato nello schema in fig. 3.1, che rappresenta le entità di dominio e le relazioni tra esse. Le entità si dividono in due gruppi concettualmente distinti: quelle che descrivono lo stato ambientale e fisico dei robot, e quelle che rappresentano gli input esterni forniti al sistema dall'operatore. Al centro dello schema si colloca *Logic*, il componente che aggrega le informazioni provenienti da entrambi i gruppi e produce come output i comandi di movimento.

Nel primo gruppo, l'entità principale è *Robot*, identificata univocamente da un proprio ID. A essa sono collegate diverse entità che ne descrivono lo stato: *Neighborhood* rappresenta l'insieme dei robot vicini, caratterizzato da un tipo e da un raggio opzionale, ed è legato al robot da una relazione di aggregazione;

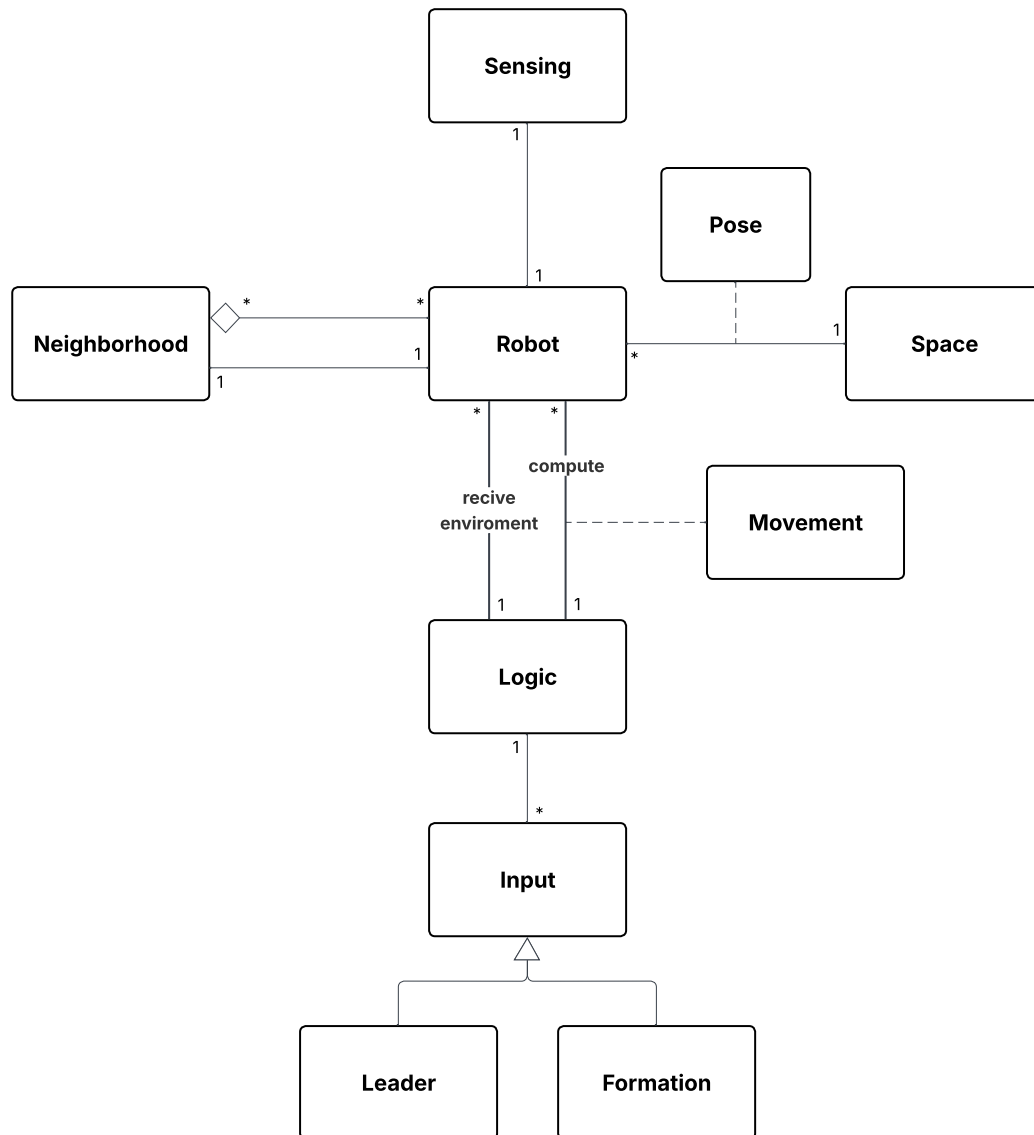


Figura 3.1: Schema delle entità di dominio.

Space è lo spazio in cui il robot si muove, e all'interno di esso il robot assume una *Pose*, composta da posizione e orientamento; infine, *Sensing* rappresenta in modo generico tutte le categorie di dati sensoriali associabili a un robot, fungendo da punto di estensione del modello per acquisizioni future. L'entità *Movement* rappresenta il comando di spostamento calcolato da *Logic* a partire dallo stato ambientale ricevuto dai robot.

Nel secondo gruppo si trovano le entità che modellano gli input operativi del sistema: *Formation*, che indica la formazione geometrica impartita allo sciame, e *Leader*, che identifica il robot designato come capo del gruppo. Entrambe sono modellate come sottotipi di una entità astratta *Input*, ricevuta da *Logic* per influenzare il comportamento aggregato del sistema.

Capitolo 4

Progettazione

Definiti nel capitolo precedente l'obiettivo, i vincoli e il modello di dominio del sistema, questo capitolo ne descrive la progettazione. Si parte da una prospettiva architetturale ampia, collocando il nuovo componente rispetto al sistema Emerge già esistente (sezione 4.1); si scende poi nel dettaglio della sua struttura interna (sezione 4.2), fino alle scelte relative a ciascun protocollo esposto (sezione 4.2.1).

4.1 Architetturale rispetto al sistema

Come anticipato nella sezione 3.1, un requisito trasversale all'intera soluzione era la necessità di separare nettamente il mondo interno del sistema da quello esterno: da un lato i moduli già sviluppati e i robot che comunicano tramite MQTT, dall'altro i client che avrebbero dovuto accedere alle stesse risorse attraverso protocolli eterogenei. Qualsiasi soluzione che non avesse rispettato questa separazione avrebbe rischiato di far collidere i due mondi, costringendo a modificare i moduli esistenti ogniqualevolta fosse necessario aggiungere o cambiare un protocollo esterno.

Tenuto conto del vincolo di non modificabilità dei moduli già sviluppati, illustrato nella sezione 3.2, la struttura generale della soluzione si è delineata in modo abbastanza naturale: era sufficiente introdurre un unico modulo aggiuntivo che si frapponesse tra l'interno e l'esterno del sistema, fungendo da punto di mediazione tra il broker MQTT interno e i client esterni. Questo modulo avrebbe assorbito

tutta la complessità legata alla molteplicità dei protocolli, lasciando invariato il comportamento degli altri componenti del sistema.

La scelta di concentrare questa responsabilità in un singolo modulo dedicato porta con sé un ulteriore vantaggio: la dashboard, attualmente costretta a ricorrere a MQTT over WebSocket per aggirare i limiti del browser, potrà migrare verso un'interfaccia più adatta in un momento successivo, senza che ciò richieda alcun intervento sul resto del sistema. Allo stesso modo, qualsiasi nuovo client o robot che utilizzi un protocollo diverso da MQTT potrà essere integrato semplicemente aggiungendo il supporto corrispondente al modulo, senza toccare il nucleo già funzionante.

La soluzione adottata prende il nome di *domain gateway*: un componente che si colloca al confine tra il sistema interno e il mondo esterno, traducendo i messaggi da un protocollo all'altro e mantenendo disaccoppiati i due lati. Nella fig. 4.1 è rappresentata l'architettura risultante a livello di sistema: il gateway si connette al broker MQTT interno come qualsiasi altro modulo, sottoscrive i topic di interesse e pubblica gli aggiornamenti ricevuti verso i client esterni attraverso i protocolli da esso esposti; in direzione opposta, i messaggi provenienti dall'esterno vengono tradotti e pubblicati sul broker interno, rendendoli accessibili ai moduli già esistenti senza che questi debbano essere a conoscenza della provenienza.

Questa scelta rispetta pienamente i vincoli definiti in fase di analisi: il broker MQTT interno rimane invariato nel suo ruolo di mezzo di comunicazione tra i moduli esistenti, e nessuno di questi richiede modifiche. Al contempo, l'architettura soddisfa il requisito di estendibilità: aggiungere in futuro il supporto a un nuovo protocollo significa intervenire esclusivamente sul gateway, senza che il resto del sistema ne sia coinvolto.

Vale la pena osservare che il disaccoppiamento introdotto dal gateway opera in entrambe le direzioni. Se da un lato isola i client esterni dal protocollo interno, dall'altro isola i moduli interni dai protocolli esposti verso l'esterno. Ne consegue che, qualora in futuro si decidesse di sostituire MQTT con un meccanismo di comunicazione interno diverso, l'intervento rimarrebbe confinato al gateway: i client esterni continuerebbero a interagire con le stesse interfacce senza alcuna modifica, esattamente come accade per i moduli interni. Questa simmetria non era un obiettivo esplicito del progetto – il requisito era superare la dipendenza verso

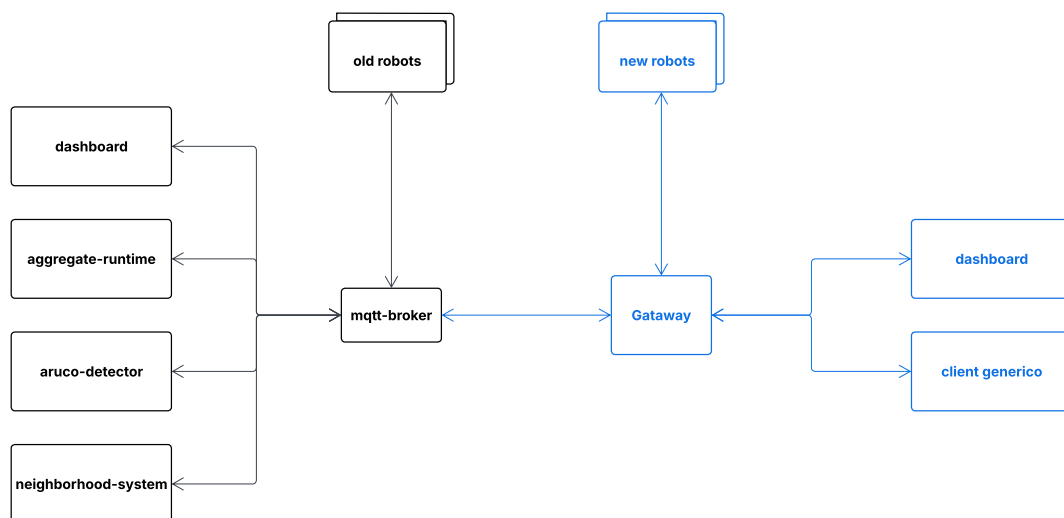


Figura 4.1: Architettura del sistema con l'introduzione del gateway. I componenti evidenziati in blu rappresentano le aggiunte introdotte.

l'esterno, non eliminare MQTT – ma emerge come proprietà naturale dell'architettura scelta, e rappresenta un valore aggiunto in ottica di evoluzione futura del sistema.

Definita l'architettura a livello di sistema, rimane da chiarire un aspetto che emerge direttamente dai requisiti funzionali: il gateway deve essere in grado di supportare sia client con un modello di interazione a notifica continua, analoga al publish/subscribe interno, sia client che interrogano le risorse in modo puntuale secondo un modello request/response. Per i protocolli event-driven il gateway si comporta semplicemente come traduttore: ogni aggiornamento ricevuto dal broker interno viene inoltrato ai client sottoscritti senza ulteriore logica. Per i protocolli request/response la situazione è diversa: quando un client richiede il valore corrente di una risorsa, il gateway deve essere in grado di rispondere immediatamente, indipendentemente dal momento in cui l'ultimo aggiornamento è stato pubblicato sul broker. Questo rende necessaria la presenza di una cache che mantenga l'ultimo valore noto per ciascuna risorsa, dalla quale il gateway possa attingere al momento della richiesta.

Poiché nessun modulo del sistema attuale offre un servizio di questo tipo, si è valutato di introdurre un secondo nuovo componente dedicato esclusivamente

alla cache oppure integrare questa funzionalità direttamente nel gateway. Considerando che lo scopo della cache è strettamente funzionale al gateway stesso – non si tratta di uno stato condiviso con altri moduli – si è ritenuto più coerente mantenerla all'interno del gateway, evitando di introdurre una dipendenza esterna per una responsabilità che rimane locale al componente.

4.2 Architetturale di dettaglio dell'applicativo

Definito il ruolo del gateway rispetto al resto del sistema, è possibile passare alla progettazione della sua struttura interna. I requisiti impongono la realizzazione di un componente multi-protocollo con due facce distinte: una rivolta verso l'interno, che dialoga con il broker MQTT, e una rivolta verso l'esterno, che espone ai client le interfacce appropriate a ciascun protocollo. Un ulteriore vincolo trasversale, emerso già in fase di analisi, prevede che l'introduzione del supporto a un nuovo protocollo non debba richiedere modifiche al nucleo del modulo, ma si traduca esclusivamente nell'implementazione di una nuova componente aggiuntiva.

L'approccio adottato per soddisfare questi requisiti si articola attorno a tre elementi principali: le *connessioni*, i *bus* di comunicazione interni e la *cache*.

Le connessioni sono sotto-moduli del gateway, ciascuno responsabile della gestione di un gruppo di interfacce esterne: ad esempio, il supporto a HTTP Representational State Transfer (REST) è incapsulato in una connessione dedicata, così come il supporto a WebSocket o a Constrained Application Protocol (CoAP). Questa scelta mantiene la complessità di ciascun protocollo confinata all'interno del sotto-modulo corrispondente, rendendo il nucleo del gateway ignaro dei dettagli di ciascuna tecnologia esposta.

I bus sono i canali attraverso cui le connessioni si scambiano messaggi senza conoscersi direttamente. Ne sono stati definiti due: l'*Inbound Bus*, che trasporta i messaggi provenienti dall'esterno verso la connessione interna responsabile di pubblicarli sul broker MQTT, e l'*Outbound Bus*, che trasporta gli aggiornamenti ricevuti dal broker verso tutte le connessioni esterne interessate. Tramite dependency injection, entrambi i bus vengono passati a ciascuna connessione al momento dell'inizializzazione: ogni connessione può pubblicare messaggi su un bus oppure iscriversi a esso, ricevendo una callback ogni volta che un altro componente vi pub-

blica. In questo modo il gateway opera come una rete di componenti disaccoppiati, coordinati esclusivamente attraverso i due canali condivisi.

Vale la pena osservare che il modello a bus non offre garanzie di consegna verso i singoli subscriber: un messaggio pubblicato viene considerato recapitato non appena è in transito sul canale, indipendentemente dal fatto che una connessione specifica lo abbia effettivamente ricevuto e processato. Questa caratteristica rende l'architettura naturalmente orientata a un modello fire-and-forget, che risulta coerente con la semantica di MQTT e dei protocolli event-driven in generale, ma che è opportuno tenere presente qualora in futuro si volessero esporre interfacce con garanzie di consegna più stringenti.

Introdotti i bus, l'inserimento della cache si ricava in modo naturale dall'architettura già definita. Come discusso nella sezione 4.1, i protocolli request/response richiedono che il gateway possa rispondere immediatamente al valore corrente di una risorsa, indipendentemente da quando è stato pubblicato l'ultimo aggiornamento. A tale scopo, la cache si iscrive all'*Outbound Bus* e aggiorna la propria snapshot a ogni messaggio in transito, costruendo una visione coerente dello stato del sistema. Come le connessioni, anche la cache viene passata tramite dependency injection ai soli sotto-moduli che la richiedono, ovvero quelli che implementano protocolli request/response.

Un aspetto rilevante riguarda il momento in cui un messaggio viene considerato valido ai fini della cache. Quando un client esterno pubblica un aggiornamento tramite una qualsiasi interfaccia del gateway, il valore non viene registrato immediatamente: il messaggio percorre prima l'*Inbound Bus*, viene pubblicato sul broker MQTT dalla connessione interna, e solo dopo che il broker lo ha ritrasmesso al gateway stesso – come farebbe con qualsiasi altro subscriber – esso transita sull'*Outbound Bus* e viene acquisito dalla cache. Questa scelta garantisce che i valori memorizzati siano sempre quelli effettivamente visibili agli altri moduli del sistema, e non stati intermedi non ancora confermati. Nella fig. 4.2 è riportata una rappresentazione schematica di questa struttura.

Come già osservato per l'architettura di sistema, anche a questo livello emerge una proprietà non esplicitamente richiesta dai requisiti, ma che rappresenta un valore aggiunto in ottica di evoluzione futura. L'adozione dei bus come unico mezzo di coordinamento tra le connessioni rende possibile, in linea di principio, affiancare

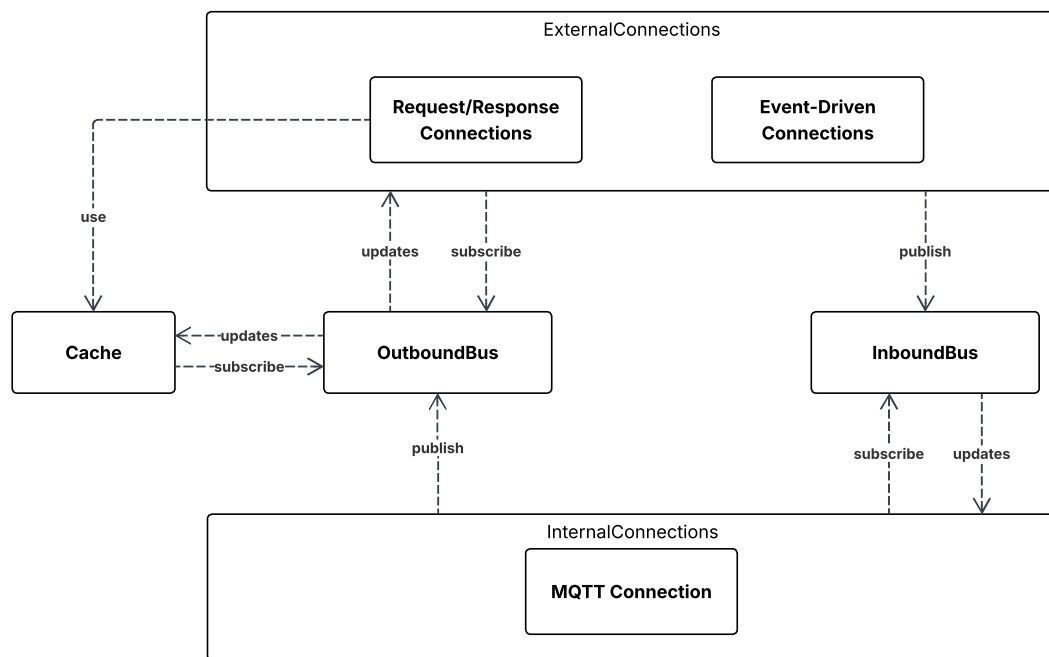


Figura 4.2: Architettura interna del gateway: connessioni, bus e cache.

alla connessione MQTT interna una seconda connessione che utilizzi un protocollo diverso, consentendo una transizione graduale verso un nuovo meccanismo di comunicazione interno senza dover modificare né le connessioni esterne né il nucleo del gateway.

4.2.1 Progettazione delle connessioni

Definita la struttura interna del gateway nei suoi elementi fondamentali – connessioni, bus e cache – è possibile scendere al livello delle singole connessioni e descrivere le scelte progettuali relative a ciascun protocollo supportato. Ogni connessione costituisce un sotto-modulo autonomo, responsabile di tradurre le primitive del proprio protocollo nelle operazioni sui bus interni, e viceversa. I protocolli esposti sono stati scelti in funzione dei profili di utilizzo individuati in fase di analisi nella sezione 3.3.3: HTTP e WebSocket per i client browser, CoAP per i dispositivi embedded, e un'interfaccia REST comune per i client generici che privilegiano la semplicità di integrazione. Le sezioni seguenti illustrano le decisioni prese per ciascuna connessione.

Connessione MQTT

La connessione MQTT rappresenta il lato interno del gateway: è l'unica connessione che interagisce direttamente con il broker, e il suo ruolo è quello di fare da ponte tra il mondo MQTT e i bus interni. Il suo funzionamento è simmetrico: in direzione *inbound* si iscrive all'*Inbound Bus* e, a ogni messaggio ricevuto, lo pubblica sul broker con il topic e il payload corrispondenti; in direzione *outbound* si sottoscrive a tutti i topic del broker e reinoltra ogni messaggio ricevuto sull'*Outbound Bus*, rendendolo disponibile a tutte le connessioni esterne e alla cache. Nonostante il suo ruolo centrale, questa connessione è strutturalmente la più semplice dell'intero gateway: non espone interfacce verso l'esterno, non interagisce con la cache e non contiene logica applicativa propria. La sua semplicità è una conseguenza diretta dell'architettura a bus: tutta la complessità di coordinamento è già assorbita dal meccanismo di pubblicazione e sottoscrizione, e la connessione MQTT si limita a operare come traduttore fedele tra il protocollo del broker e il canale interno.

Connessione HTTP

La connessione HTTP espone le risorse del sistema attraverso un'interfaccia REST, organizzata attorno alle entità individuate in fase di analisi nella sezione 3.4. Per ciascuna entità sono stati definiti due endpoint: uno in lettura, accessibile tramite il metodo `GET`, che restituisce il valore corrente della risorsa attingendo dalla cache, e uno in scrittura, accessibile tramite il metodo `PUT`, che consente a un client esterno di aggiornare lo stato di una risorsa pubblicando il nuovo valore sull'*Inbound Bus*. È stato scelto `PUT` e non `POST`: `POST` implica convenzionalmente la creazione di una nuova risorsa, mentre `PUT` esprime l'aggiornamento di una risorsa già identificata dal suo Uniform Resource Identifier (URI). Poiché le entità del sistema esistono indipendentemente dal gateway – sono i robot e i moduli interni a definirne il ciclo di vita – un'operazione di scrittura esterna non crea nulla, ma aggiorna un valore già presente nel dominio. L'uso di `PUT` riflette quindi in modo più preciso la semantica dell'operazione, e rimane valido anche nel caso in cui la risorsa non abbia ancora pubblicato alcun aggiornamento: il client la tratta comunque come una risorsa esistente di cui sta modificando lo stato. L'interfaccia esposta dalla connessione è documentata tramite uno schema OpenAPI Specification (OpenAPI) [Ope21], che descrive in modo formale gli endpoint disponibili, i formati di input e output attesi e i codici di risposta previsti.

Connessione CoAP

La connessione CoAP ricalca nella struttura quella HTTP: anche in questo caso vengono esposti endpoint in lettura e in scrittura per ciascuna risorsa, con una semantica analoga a quella già descritta per i metodi `GET` e `PUT`. CoAP è stato progettato fin dall'origine come un protocollo REST ottimizzato per ambienti con risorse computazionali e di rete limitate, riprendendo il modello request/response di HTTP e adattandolo a un trasporto User Datagram Protocol (UDP) con overhead ridotto.

Rispetto a HTTP, tuttavia, CoAP offre un meccanismo aggiuntivo che risulta particolarmente utile nel contesto del gateway: la modalità *Observe*, definita nel Request for Comments (RFC) 7641 [Har15]. Tramite Observe, un client può registrare il proprio interesse verso una risorsa inviando una richiesta `GET` arricchita

di un'apposita opzione; da quel momento, il server non si limita a rispondere una volta sola, ma continua a notificare il client ogni volta che il valore della risorsa cambia, senza che sia necessaria una nuova richiesta. Questo trasforma CoAP da protocollo puramente request/response a un protocollo ibrido, capace di supportare anche un modello di interazione event-driven analogo a quello del broker MQTT interno. Per il gateway, l'integrazione di Observe si traduce nel seguente comportamento: quando un client si registra come osservatore di una risorsa, a ogni aggiornamento ricevuto, la connessione invia una notifica al client registrato.

Connessione WebSocket

La progettazione della connessione basata su TCP ma event-based ha richiesto una riflessione più ampia rispetto alle altre. L'obiettivo era ottenere, per i client che comunicano tramite protocolli basati su Transmission Control Protocol (TCP), un meccanismo analogo a quello offerto dalla modalità Observe di CoAP: la possibilità di ricevere aggiornamenti continui sulle risorse senza dover interrogare il sistema a ogni cambiamento. Prima di giungere alla scelta definitiva sono state valutate altre due alternative, descritte di seguito insieme alle ragioni che hanno portato a scartarle.

Callback Il meccanismo delle callback prevede che il client, al momento della sottoscrizione a una risorsa, fornisca al gateway un URI a cui inviare le notifiche. Il gateway, a ogni aggiornamento, effettua una richiesta HTTP verso quell'indirizzo, trasportando il nuovo valore come payload. Lo schema di interazione è illustrato nella fig. 4.3.

Questa soluzione è stata rapidamente esclusa per due ragioni principali. In primo luogo, avrebbe richiesto che ogni client esponga a sua volta un server HTTP raggiungibile dal gateway: un requisito che esclude dispositivi con risorse limitate, come alcuni robot dell'ecosistema IoT di riferimento, e impone un onere architetturale significativo su tutti i client. In secondo luogo, ogni notifica comporta la creazione di una nuova connessione TCP, poiché le richieste HTTP tradizionali non sono persistenti: in un sistema IoT in cui gli aggiornamenti possono essere frequenti e numerosi, l'overhead derivante dall'apertura e chiusura continua di connessioni risulta inaccettabile sia lato gateway che lato client.

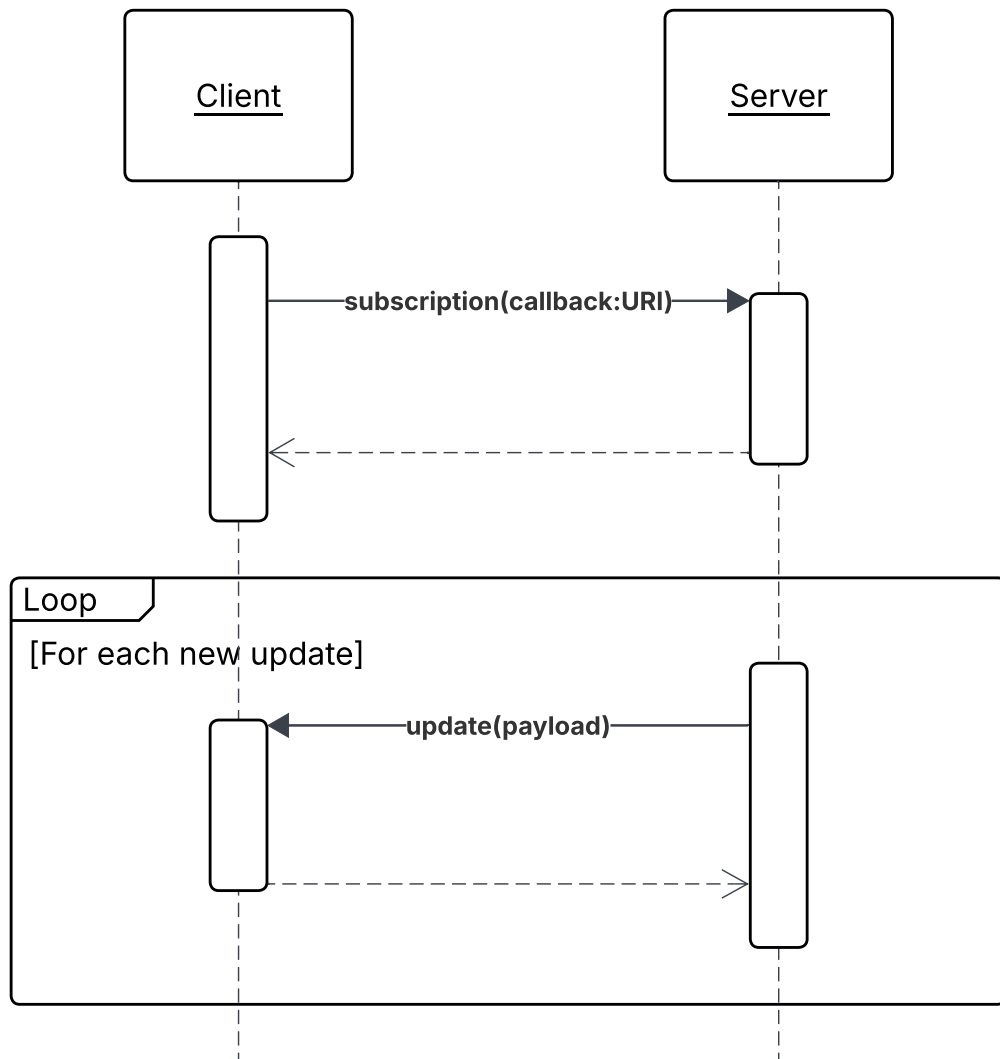


Figura 4.3: Diagramma di sequenza del meccanismo a callback. Gli aggiornamenti non sono sincroni. Gli aggiornamenti lato client non sono riportati, poiché equivarrebbero a una semplice chiamata HTTP.

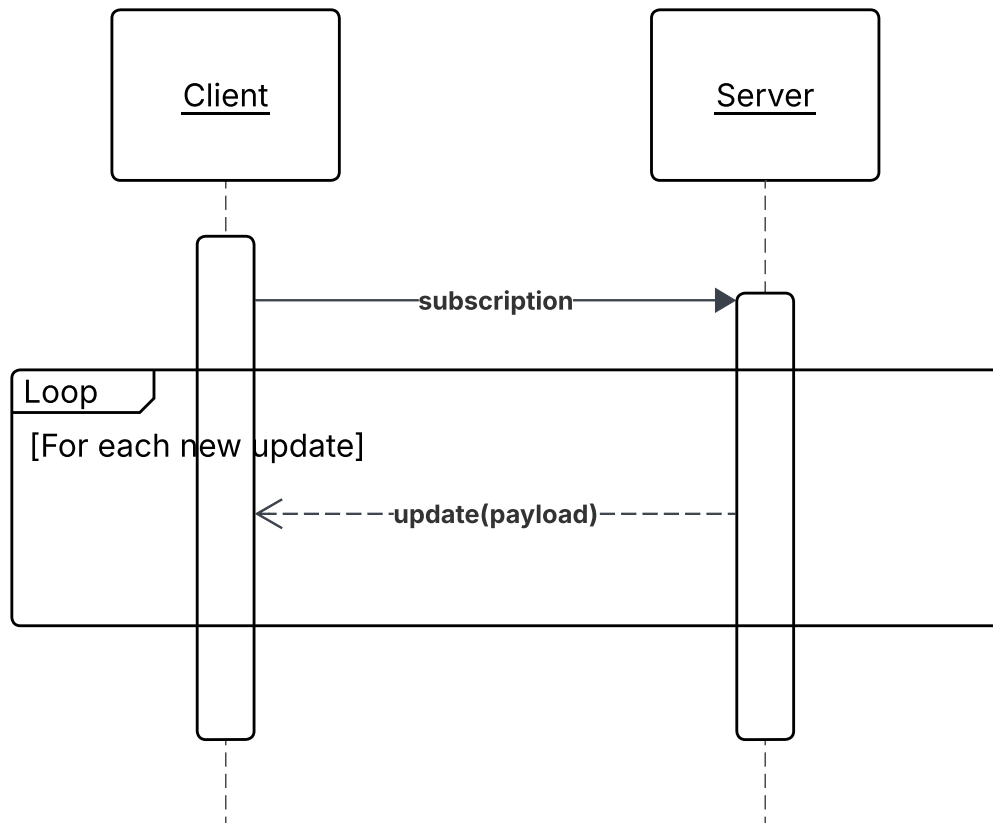


Figura 4.4: Diagramma di sequenza del meccanismo a Server-Sent Events. Come per il caso precedente, gli aggiornamenti non sono sincroni e quelli lato client non sono riportati.

Server-Sent Events I Server-Sent Events (SSE) rappresentano un'alternativa più adatta al contesto: il client apre una connessione HTTP persistente verso il gateway e riceve gli aggiornamenti in modo unidirezionale, senza dover esporre alcun endpoint. Il server può inviare eventi in qualsiasi momento lungo lo stesso canale già aperto, eliminando il problema della creazione ripetuta di connessioni. Lo schema di interazione è illustrato nella fig. 4.4.

Un ulteriore vantaggio di questa soluzione sarebbe stata la sua somiglianza con l'interfaccia CoAP già descritta: le due connessioni avrebbero condiviso una semantica di utilizzo pressoché identica. Nonostante questi pregi, anche questa

strada è stata abbandonata per due motivi strettamente correlati. Il primo riguarda la gestione delle sottoscrizioni a risorse multiple: un client che avesse voluto ricevere aggiornamenti su più risorse contemporaneamente sarebbe stato costretto a mantenere una connessione HTTP persistente distinta per ciascuna di esse, con un dispendio di risorse proporzionale al numero di risorse osservate. Il secondo motivo riguarda la direzione di scrittura: i SSE sono intrinsecamente unidirezionali, quindi ogni aggiornamento che il client avesse voluto inviare al sistema avrebbe richiesto comunque l'apertura di una nuova connessione HTTP, ricorrendo all'endpoint `PUT` della connessione HTTP già descritta nella sezione 4.2.1. La combinazione di queste due limitazioni porta a una proliferazione di connessioni parallele non strettamente necessarie.

WebSocket Per superare entrambe le limitazioni dei SSE, si è scelto di adottare WebSocket, un protocollo che stabilisce un canale bidirezionale full-duplex su un'unica connessione TCP persistente. Su questo canale transitano sia i messaggi inviati dal client verso il sistema sia le notifiche inviate dal gateway verso il client, relative alle risorse di interesse. Lo schema di interazione è illustrato nella fig. 4.5.

Poiché WebSocket non definisce un meccanismo nativo di filtraggio dei messaggi per risorsa, è stato necessario progettare un sistema di sottoscrizione dedicato. Tale sistema è costruito a supporto dell'unico endpoint WebSocket esposto dalla connessione e si compone di tre endpoint REST aggiuntivi, utilizzati esclusivamente per la gestione del ciclo di vita delle sottoscrizioni: un endpoint `POST` per la creazione, un endpoint `GET` per la lettura e un endpoint `DELETE` per l'eliminazione.

Il flusso di utilizzo è il seguente: il client invia una richiesta `POST` specificando la risorsa per cui desidera ricevere aggiornamenti; in risposta ottiene un identificatore univoco di sottoscrizione. Una volta stabilita la connessione WebSocket, il client invia quell'identificatore lungo il canale aperto: la connessione lo riceve e associa la sottoscrizione alla sessione WebSocket del client. Per aggiungere ulteriori risorse di interesse è sufficiente ripetere il processo – creare una nuova sottoscrizione tramite `POST` e inviare il relativo identificatore sulla stessa connessione già aperta – senza che sia necessario aprire canali aggiuntivi. Da quel momento in poi, ogni aggiornamento relativo alla risorsa sottoscritta viene notificato al client attraverso

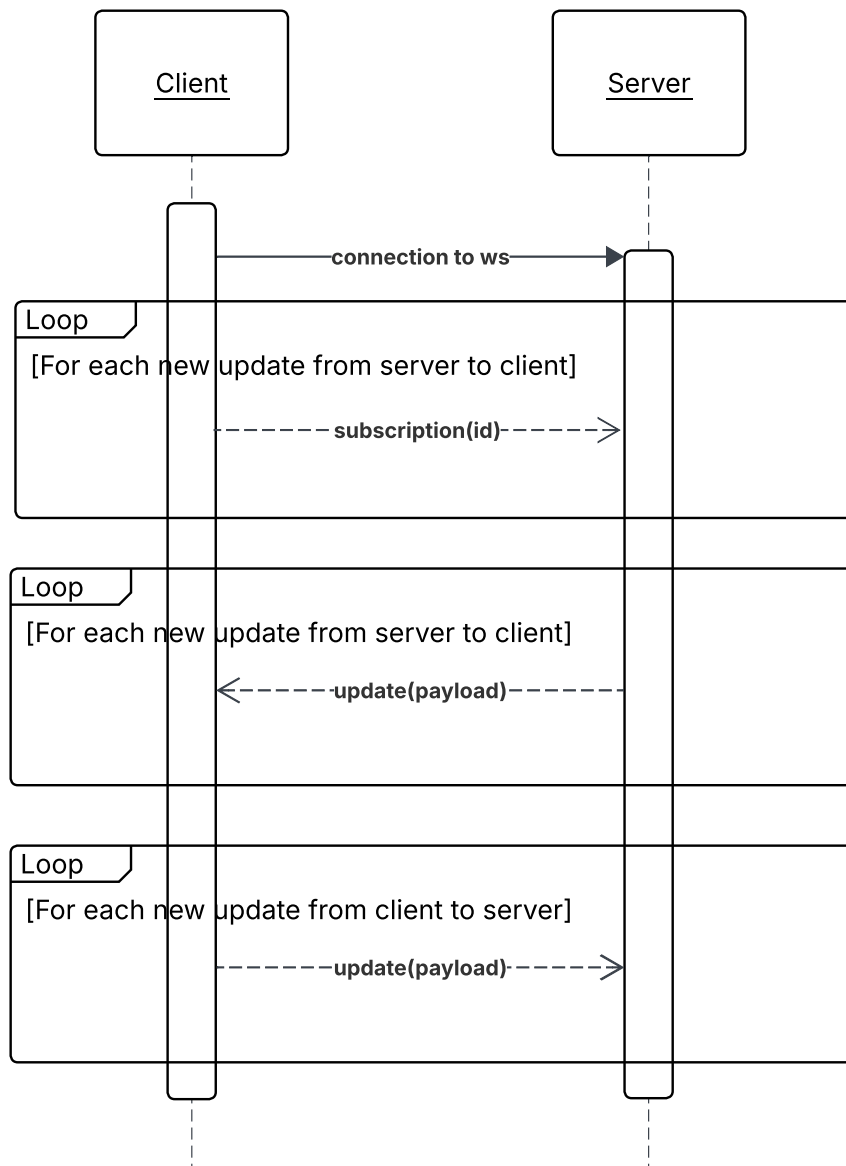


Figura 4.5: Diagramma di sequenza del meccanismo a WebSocket. Come per i casi precedenti, gli aggiornamenti non sono sincroni. I tre blocchi non sono necessariamente sequenziali: lo schema fornisce una rappresentazione puramente schematica dell'interazione.

la connessione persistente già stabilita, senza che sia necessaria alcuna ulteriore richiesta.

Questa scelta architetture è motivata da tre ragioni principali. In primo luogo, la *separazione delle responsabilità*: REST è semanticamente adatto alla gestione del ciclo di vita delle risorse, mentre il canale WebSocket rimane dedicato esclusivamente al flusso di dati in tempo reale. In secondo luogo, l'*idempotenza e affidabilità*: le richieste REST possono essere ritentate in caso di errore di rete senza ambiguità, laddove un messaggio di sottoscrizione perso sul canale WebSocket richiederebbe logica di retry e acknowledgment aggiuntiva. Infine, il *disaccoppiamento dalla connessione*: la sottoscrizione viene creata indipendentemente dall'apertura del canale WebSocket, rendendo il processo di riconnessione trasparente — il client riapre il canale e reinvia gli identificatori già noti, senza dover ricostruire lo stato delle sottoscrizioni.

Questa architettura consente a un singolo client di gestire un numero arbitrario di sottoscrizioni su risorse diverse tramite un'unica connessione WebSocket, riducendo al minimo le risorse di rete impiegate.

Capitolo 5

Implementazione

Conclusa la fase di progettazione, questo capitolo descrive come le scelte architetture discusse nel capitolo precedente sono state effettivamente realizzate, seguendo la medesima scomposizione. Si parte dallo stack tecnologico adottato e dall'organizzazione del codice (sezione 5.1), per poi approfondire le decisioni implementative relative ai componenti del nucleo del gateway (sezione 5.2) e infine scendere al dettaglio di ciascuna connessione protocollare (sezione 5.2.1).

5.1 Stack tecnologico e organizzazione del codice

Definita l'architettura del gateway nei suoi elementi essenziali, questa sezione ne descrive la realizzazione concreta: il linguaggio e gli strumenti adottati, le librerie su cui si appoggiano le singole connessioni, e il modo in cui il codice è organizzato.

Linguaggio e gestione delle dipendenze Il gateway è stato sviluppato in Python 3.13, scelto per il supporto nativo ad `asyncio`. Trattandosi di un componente la cui natura è quasi interamente di I/O – richieste HTTP, sessioni WebSocket, datagrammi CoAP e le due direzioni della connessione MQTT – un modello a singolo processo e singolo event loop, in cui ogni operazione di rete è una coroutine concorrente, è risultato ideale a soddisfare il requisito di responsività sotto carico senza dover introdurre concorrenza basata su thread o processi multipli, i quali avrebbero aumentato notevolmente la complessità della soluzione. La gestione del-

le dipendenze e del processo di build è affidata a `uv` [uv26], scelto principalmente per la sua velocità.

Librerie principali Attorno a Python gravitano poche librerie, ciascuna responsabile di un solo protocollo o di una sola responsabilità trasversale e capace di fornire un servizio asincrono. FastAPI [Ram] è stata scelta per le connessioni HTTP e WebSocket poiché le supporta entrambe all'interno della stessa applicazione Asynchronous Server Gateway Interface (ASGI), eseguita tramite il server Uvicorn [TC], generando automaticamente la documentazione OpenAPI prevista in fase di progettazione direttamente dalle firme dei router e dai modelli Pydantic [CJR⁺26], e offrendo un meccanismo di dependency injection – riutilizzato per il controllo di salute descritto più avanti – senza richiedere codice aggiuntivo. Per CoAP è stata adottata `aiocoap` [At], l'implementazione asincrona più matura disponibile per Python, capace di fornire supporto nativo alla modalità Observe. La connessione MQTT si appoggia su `aiomqtt` [emp26] libreria asincrona che espone un'interfaccia tramite `async/await` invece che tramite callback. Pydantic, infine, attraversa trasversalmente tutte le connessioni: ogni entità del modello di dominio definito in fase di analisi è un modello Pydantic, usato sia per validare e serializzare i payload MQTT sia per i corpi delle richieste HTTP, i frame WebSocket e i payload CoAP, costituendo un'unica fonte di verità per lo schema dei dati indipendentemente dal protocollo con cui transitano.

Containerizzazione e configurazione Il gateway viene distribuito tramite un'immagine Docker costruita in due fasi: una fase di build, in cui `uv` installa le dipendenze bloccate dal lock file e l'interprete Python pinnato, e una fase finale, basata su un'immagine Debian minimale ed eseguita da un utente non privilegiato, che si limita a copiare l'ambiente virtuale già pronto. Il Dockerfile documenta le porte associate ai protocolli di rete coinvolti, in particolare quella su cui è presente il server ASGI che ospita le connessioni HTTP e WebSocket e quella UDP per il server CoAP. La configurazione – l'indirizzo del broker MQTT e, opzionalmente, l'indirizzo su cui deve essere avviato il server CoAP – è esternalizzata tramite variabili d'ambiente o file `.env`, letta da una classe di impostazioni basata su Pydantic; quest'ultima opzione si è rivelata necessaria perché `aiocoap` non è in grado

di effettuare il bind predefinito su macOS, e va quindi sovrascritta manualmente in quell'ambiente.

Organizzazione dei moduli Prima di descrivere nel dettaglio le singole decisioni implementative, è utile delineare brevemente il contesto generale: il pacchetto principale si articola in tre rami, oltre al file `main`.

- `/core`: contiene le interfacce e le classi principali, derivate direttamente dalle considerazioni della sezione 4.2, tra cui `Connection`, `Bus`, `Cache`, `Container` e `Monitor`, quest'ultimo descritto più dettagliatamente in seguito.
- `/connections`: raccoglie le implementazioni di tutte le connessioni.
- `/models/topics`: raccoglie, tramite classi `Pydantic`, tutti i modelli e i relativi payload delle entità di dominio.

5.2 Principali decisioni implementative

Definita l'organizzazione dei moduli, è possibile addentrarsi nelle scelte implementative relative ai componenti raccolti nel pacchetto `/core`, ciascuno dei quali dà forma concreta alle astrazioni discusse nel capitolo di progettazione. I primi tre paragrafi – `Connection`, `Bus` e `Cache` – riprendono nell'ordine gli elementi fondamentali dell'architettura interna del gateway già introdotti nella sezione 4.2; il quarto, `Container`, descrive come queste tre astrazioni vengono concretamente istanziate e collegate tra loro al momento dell'avvio dell'applicazione; il quinto e ultimo, `Monitor`, introduce infine un componente trasversale non previsto in fase di progettazione, nato dall'esigenza di esporre in modo centralizzato lo stato di salute del gateway. Esauriti questi aspetti comuni a tutto il gateway, la trattazione scende infine al livello delle singole connessioni (sezione 5.2.1), ricalcando lo stesso passaggio dal generale al particolare già adottato in sezione 4.2.1 per descrivere, protocollo per protocollo, le scelte progettuali corrispondenti.

Connection La classe astratta `Connection`, definita in `core/connection.py`, costituisce il punto di estensione del gateway descritto nella sezione 4.2: ogni

sotto-modulo che implementa un protocollo, sia esso rivolto verso l'esterno o verso l'interno, eredita da questa classe e ne implementa i due metodi astratti `start()` e `stop()`, che ne definiscono il ciclo di vita. Il costruttore riceve tramite dependency injection i riferimenti a *Inbound Bus* e *Outbound Bus*, rendendoli disponibili alle sottoclassi senza che queste debbano occuparsi della loro creazione. Questa interfaccia minimale è sufficiente a garantire l'uniformità richiesta. Ogni connessione, indipendentemente dal protocollo che implementa, può essere avviata e arrestata in modo identico dal resto del sistema. Lo stesso contratto è rispettato anche da `ExternalConnections` e `InternalConnections`, due classi composite che a loro volta implementano `Connection` ma delegano l'effettiva implementazione di `start()` e `stop()` all'insieme delle connessioni che aggregano, invocandole concorrentemente tramite `asyncio.gather`. Questa scelta consente al punto di ingresso dell'applicazione, descritto più avanti in `Container`, di rimanere del tutto ignaro del numero e della natura delle connessioni effettivamente attive: esso avvia e arresta due soli gruppi, indipendentemente da quanti protocolli essi contengano al loro interno.

Bus Coerentemente con quanto discusso nella sezione 4.2, il bus è realizzato attraverso una classe astratta `Bus`, che espone i due soli metodi `publish()` e `subscribe()`, e un'unica implementazione concreta, `MessageBus`, riutilizzata per entrambe le istanze – *Inbound* e *Outbound* – create una sola volta in `Container`. `MessageBus` realizza una semantica fire-and-forget: ogni chiamata a `publish()` itera sui subscriber registrati e, per ciascuno di essi, schedula una coroutine indipendente tramite `asyncio.create_task`, restituendo immediatamente il controllo al chiamante senza attendere che la notifica sia stata effettivamente processata. Ogni invocazione di un subscriber è inoltre incapsulata in un blocco `try/except` che intercetta e registra qualsiasi eccezione, impedendo che il fallimento di un singolo subscriber comprometta gli altri o il publisher stesso. Questa combinazione di concorrenza e isolamento dei fallimenti è ciò che permette al bus di rimanere non bloccante anche quando uno o più subscriber risultano lenti o temporaneamente non disponibili, a costo di rinunciare a garanzie di ordine o di consegna — un compromesso già discusso e ritenuto accettabile in fase di progettazione.

Cache La classe astratta **Cache** incapsula la logica di sottoscrizione descritta nella sezione 4.2: il proprio costruttore riceve l'*Outbound Bus* e vi si registra immediatamente come subscriber, cosicché ogni sua sottoclasse erediti automaticamente il comportamento di aggiornamento senza doverlo reimplementare. Le sottoclassi devono limitarsi a fornire i due metodi `get()` e `_set()`, rispettivamente per la lettura e per la scrittura nel proprio backing store, oltre ai metodi di ciclo di vita `start()` e `stop()`, predisposti per backend che richiedano un'inizializzazione esplicita – ad esempio l'apertura di una connessione di rete verso un database esterno – ma lasciati come no-op nella base. L'unica implementazione attualmente presente, **InMemoryCache**, si appoggia a un semplice dizionario Python indicizzato per topic. La scelta di mantenere l'interfaccia **Cache** separata dalla sua implementazione, per quanto in questo caso possa apparire eccessivo rispetto agli obiettivi del progetto, riflette lo stesso principio di disaccoppiamento adottato per le connessioni. Sostituire **InMemoryCache** con un'implementazione distribuita, ad esempio basata su Redis, per supportare repliche multiple del gateway, richiederebbe di intervenire su un'unica riga in **Container**, senza alcun impatto sulle connessioni che la utilizzano.

Container Tutte le classi concrete del gateway vengono istanziate e collegate fra loro in un unico punto, la classe **Container**, che funge da composition root dell'applicazione. Nel proprio costruttore, **Container** crea le due istanze di **MessageBus**, la **InMemoryCache**, l'**HealthMonitor** (descritto nel paragrafo successivo) e il router FastAPI condiviso, per poi passarli, tramite constructor injection, ai due gruppi **InternalConnections** ed **ExternalConnections**. Nessun componente del gateway accede a uno stato condiviso globale o importa un singleton al di fuori di questo grafo di dipendenze: ogni classe dichiara esplicitamente, nella propria firma, tutto ciò di cui ha bisogno, rendendo la struttura delle dipendenze leggibile direttamente dal codice. La funzione `create_app()`, definita in `main.py`, riceve un'istanza già completamente configurata di **Container** e restituisce l'applicazione FastAPI risultante, senza possedere alcuna conoscenza di come i componenti siano stati effettivamente costruiti. Questa separazione tra costruzione e utilizzo è ciò che rende agevole il testing: la suite di test costruisce un proprio **Container**, sostituendo semplicemente **MQTTConnection** con un *mock* che simula il comportamento

del broker, senza dover replicare o alterare il resto del cablaggio applicativo.

Monitor L'ultimo componente del nucleo, **HealthMonitor**, nasce dall'esigenza di esporre lo stato di salute del gateway senza che questa responsabilità debba essere gestita autonomamente da ciascuna connessione. Ogni connessione che desidera essere monitorata si registra presso l'**HealthMonitor** indicando se il proprio corretto funzionamento è da considerarsi critico per il sistema nel suo complesso; in cambio riceve un **HealthHandle**, un oggetto a sola scrittura che espone esclusivamente il metodo `report()`. Questa asimmetria è intenzionale: la connessione può comunicare il proprio stato, **UP** o **DOWN**, ma non può leggere né influenzare quello delle altre, né necessita di sapere se essa stessa sia stata registrata come critica. Il monitor aggrega questi stati in due modi: `is_healthy()` restituisce un valore booleano complessivo, vero solo se tutte le connessioni critiche risultano attualmente **UP**, mentre `snapshot()` restituisce una struttura dettagliata serializzabile in JSON con lo stato di ciascuna connessione registrata. Nel gateway, la connessione MQTT è registrata come critica, poiché un suo malfunzionamento isola completamente il gateway dal dominio interno, mentre la connessione CoAP è registrata come non critica. Lo stato viene infine esposto alla chiamata agli endpoint HTTP, sfruttando il meccanismo di dependency injection di FastAPI per restituire un errore 503 qualora una connessione critica risultasse non disponibile.

5.2.1 Implementazione delle Connessioni

Questa parte descrive come le connessioni progettate in sezione 4.2.1 sono state concretamente implementate. Prima di addentrarsi nel dettaglio delle singole connessioni, viene illustrato lo schema generale che ne accomuna l'organizzazione del codice, comune a tutti i protocolli; seguono poi, in altrettanti paragrafi dedicati, le scelte implementative specifiche di MQTT, HTTP, CoAP e WebSocket.

Schema generale Il pacchetto `/connections` rispecchia direttamente la distinzione discussa in sezione 4.1 tra connessioni interne ed esterne: si articola in due sotto-pacchetti, `externals` e `internals`, ciascuno contenente la propria classe composita (`ExternalConnections` e `InternalConnections`, già descritte nella

sezione precedente) e una cartella `connections/` in cui risiede l'implementazione di ciascun protocollo. All'interno di ciascuna di queste cartelle, ogni protocollo occupa un proprio sotto-pacchetto autonomo (`mqtt/`, `http/`, `coap/`, `websocket/`), composto da un file `connection.py` – che contiene la sottoclasse di `Connection` e ne rappresenta il punto di ingresso – ed eventuali cartelle `routers/`, `models/` o `resources/` proprie del protocollo. Questa simmetria nella struttura delle cartelle rende immediatamente riconoscibile, a chiunque esplori il codice, dove cercare l'implementazione di un dato protocollo, e mantiene indipendenti i sotto-pacchetti.

Le due connessioni costruite su FastAPI, ossia HTTP e WebSocket, condividono inoltre un meccanismo di composizione dei router: ciascuna riceve, tramite dependency injection, un riferimento al medesimo `APIRouter` – creato una sola volta in `Container` e definito come `root_router` – sul quale registra i propri endpoint tramite `include_router()`. In questo modo, l'applicazione FastAPI, istanziata in `main.py`, non deve conoscere quali protocolli siano effettivamente attivi: si limita a montare un unico router già popolato. La connessione CoAP, non basata su ASGI, segue invece un meccanismo di composizione analogo ma indipendente, costruendo un oggetto `Site` di `aiocoap` popolato con le risorse di propria competenza.

Trasversalmente a tutte le connessioni, infine, i modelli Pydantic definiti in `models/topic/payloads.py` – già introdotti nella sezione 5.1 – costituiscono l'unica rappresentazione delle entità di dominio utilizzata: la stessa classe `RobotPosition`, ad esempio, valida il payload MQTT in arrivo dal broker, il corpo di una richiesta PUT HTTP, il payload di un messaggio WebSocket e quello di una richiesta CoAP. Questo evita qualunque duplicazione di schema tra i protocolli e garantisce che un'eventuale modifica a un'entità si propaghi automaticamente a tutte le connessioni che la utilizzano.

MQTT La connessione MQTT, `MQTTConnection`, è realizzata attorno a due coroutine concorrenti e indipendenti, `_listener` e `_publisher`, avviate come task separati al momento dello `start()`. Il `_listener` apre una sessione verso il broker tramite `aiomqtt.Client`, si sottoscrive al wildcard `/#` per ricevere ogni messaggio pubblicato, e per ciascun messaggio in arrivo invoca `_dispatch`, che a sua volta tenta di risolvere il topic verso la corrispondente classe Pydantic attraverso `resolve_payload_class` – la funzione definita in `models/topic/mappings.py` che

confronta il topic contro un insieme di pattern regolari – e di validare il payload. Un topic non riconosciuto o un payload che fallisce la validazione vengono scartati con un messaggio di log, senza interrompere il flusso degli altri messaggi; in caso di successo, il payload tipizzato viene pubblicato sull'*Outbound Bus*. Il `_publisher`, simmetricamente, si sottoscrive all'*Inbound Bus* tramite il metodo `_update` e si limita ad accodare ogni coppia topic/payload ricevuta in una `asyncio.Queue` interna; un ciclo separato consuma questa coda e pubblica ciascun messaggio sul broker tramite lo stesso client MQTT. La presenza della coda disaccoppia la velocità con cui l'*Inbound Bus* consegna i messaggi da quella con cui il broker riesce effettivamente ad accettarli.

Entrambi i task condividono la stessa strategia di resilienza: un qualunque errore di connessione – broker irraggiungibile, connessione interrotta – viene intercettato, riportato come stato `DOWN` tramite l'`HealthHandle` ricevuto in costruzione, e seguito da un nuovo tentativo dopo un'attesa che cresce esponenzialmente da `BASE_RECONNECT_DELAY` (2 secondi) fino a un tetto massimo di `MAX_RECONNECT_DELAY` (60 secondi), con l'aggiunta di un piccolo ritardo casuale (*jitter*) per evitare che più istanze del gateway – nel caso fossero necessarie – tentino la riconnessione esattamente nello stesso istante.

HTTP La connessione HTTP, `HTTPConnection`, non possiede stato né task in background: la sua interezza si esaurisce nel costruttore, dove vengono istanziati e registrati sul `root_router` due router, `RobotRouter` e `ComputingRouter`, corrispondenti rispettivamente alle entità legate ai singoli robot e a quelle di input dello sciame descritte in sezione 3.4. I metodi `start()` e `stop()` risultano pertanto no-op: il ciclo di vita effettivo degli endpoint è già governato da FastAPI una volta che l'applicazione viene avviata. Ogni router segue lo stesso schema per ciascuna entità che espone: un endpoint `GET` che legge dalla cache e restituisce 404 se la risorsa non è ancora mai stata pubblicata, e un endpoint `PUT` che valida il corpo della richiesta – grazie al binding automatico di FastAPI verso il modello Pydantic dichiarato come tipo del parametro – e lo pubblica sull'*Inbound Bus*, restituendo un oggetto di risposta che riporta sia il topic di destinazione sia il payload effettivamente inoltrato, utile al chiamante per confermare l'esito dell'operazione.

A ogni endpoint viene iniettata una dipendenza tramite FastAPI: una funzione

`check_health`, definita internamente a `HTTPConnection`, che interroga l'istanza ricevuta come parametro `health_checkable` – tipizzata come `HealthReporter`, l'interfaccia a sola lettura descritta nella sezione precedente – e solleva un'eccezione 503 corredata dello snapshot dettagliato qualora una connessione critica risulti non disponibile. In questo modo, un client HTTP che tenti di leggere o scrivere una risorsa mentre, ad esempio, la connessione MQTT è temporaneamente caduta riceve immediatamente un errore esplicito, piuttosto che un valore di cache potenzialmente non più aggiornato. La documentazione OpenAPI, generata automaticamente da FastAPI a partire dalle firme dei router e dai modelli di richiesta e risposta, include anche un esempio di tale risposta di errore.

CoAP La connessione CoAP è strutturalmente la più articolata tra quelle esterne, poiché `aiocoap` non offre un livello di routing dichiarativo paragonabile a quello di FastAPI: ogni risorsa va costruita come sottoclasse di `resource.Resource` (o `resource.ObservableResource`, per supportare `Observe`) e registrata esplicitamente in un `Site` al momento dello `start()`. Per le risorse di input dello sciame, statiche e in numero fisso, sono state definite due classi dedicate, `FormationResource` e `LeaderResource`, entrambe derivate da una base comune `BaseComputingResource` che fattorizza la logica di lettura, scrittura e osservazione condivisa tra le due. Per le risorse legate ai robot, il cui numero non è noto a priori, si è invece optato per un'unica `RobotsSubsite`, registrata sul prefisso `/robots` e capace di risolvere dinamicamente, a partire dal path della richiesta, sia l'identificativo del robot sia l'endpoint richiesto (`position`, `movement`, `neighbors` o `sensing`) tramite la tabella `_ROBOT_ROUTES`; aggiungere un nuovo robot non richiede quindi alcuna modifica al codice, mentre aggiungere un nuovo tipo di endpoint richiede solo un'estensione di quella tabella.

Il supporto a `Observe`, come descritto in sezione 4.2.1, è realizzato facendo sottoscrivere ciascuna risorsa all'*Outbound Bus* fin dal proprio costruttore: ogni volta che transita un messaggio il cui topic corrisponde a quello di competenza della risorsa, questa richiama `trigger()` su ciascuna `ServerObservation` attualmente registrata, delegando ad `aiocoap` l'invio della notifica al client osservatore. La gestione delle sottoscrizioni differisce leggermente tra le due famiglie di risorse: `BaseComputingResource`, occupandosi di un solo topic, mantiene un unico insie-

me di osservazioni, mentre `RobotsSubsite`, dovendo distinguere tra topic diversi sullo stesso path radice, mantiene un dizionario che associa ciascun topic al proprio insieme di osservatori. In entrambi i casi, una richiesta `GET` su una risorsa il cui valore non è ancora presente in cache restituisce una risposta `2.05 Content` con corpo vuoto anziché `4.04 Not Found`, cosicché l'eventuale registrazione `Observe` associata rimanga valida in attesa del primo aggiornamento e il client non termini la connessione: molti client `CoAP`, infatti, interrompono l'osservazione non appena ricevono una risposta `4.04 Not Found`, anche se la richiesta `GET` era stata effettuata correttamente in modalità `Observe`.

WebSocket L'implementazione della connessione `WebSocket` è incentrata su un'unica classe, `WebSocketService`, che funge sia da gestore delle connessioni attive sia da gestore del ciclo di vita delle sottoscrizioni, ruoli espressi nel codice tramite due `Protocol` distinti, `WebSocketManager` e `SubscriptionManager`, di cui `WebSocketService` è l'unica implementazione concreta. Gli endpoint REST per la creazione, lettura ed eliminazione delle sottoscrizioni, descritti in sezione 4.2.1, sono realizzati da `WebsocketSubscriptionRouter` e delegano interamente alla parte `SubscriptionManager` del servizio. L'endpoint `WebSocket` vero e proprio, esposto da `WebsocketEndpointRouter`, delega invece al metodo `add()`, che accetta la connessione ed entra in un ciclo di ricezione fino alla disconnessione del client o a un errore di validazione. Ogni frame ricevuto viene interpretato tramite un `TypeAdapter` `Pydantic` configurato su un'unione discriminata di due soli tipi di messaggio: un `SubscriptionMessage`, che lega l'identificativo di una sottoscrizione precedentemente creata alla connessione `WebSocket` corrente – popolando il dizionario `_subscription_websocket_dict` – e un `PublishMessage`, che consente al client di scrivere direttamente una nuova entità sull'*Inbound Bus* senza dover aprire una richiesta `PUT` separata. In direzione opposta, il servizio si sottoscrive all'*Outbound Bus* al momento dello `start()`: per ogni messaggio in arrivo il cui topic compare tra quelli sottoscritti, viene inviato un `SubscriptionUpdateMessage` a ciascun `WebSocket` attualmente legato a una sottoscrizione di quel topic, con gli invii eseguiti concorrentemente e gli eventuali fallimenti di singole connessioni isolati, in modo che la disconnessione di un client non comprometta la consegna agli altri. Infine, ogni connessione `WebSocket` attiva viene tracciata come task in

un insieme dedicato, cancellato in blocco al momento dello `stop()`, garantendo uno spegnimento ordinato di tutte le sessioni aperte.

Capitolo 6

Valutazione

6.1 Testing automatico

Definita l'architettura interna del gateway e illustrate le scelte implementative relative a ciascuna connessione, questa sezione descrive come tali componenti siano stati verificati. Come anticipato tra i requisiti non funzionali in sezione 3.3.2, il sistema doveva essere progettato in modo da facilitare il testing automatico: la struttura a **Container** descritta in sezione 5.2, in cui ogni dipendenza viene dichiarata esplicitamente e iniettata dall'esterno, è proprio ciò che rende possibile costruire, in fase di test, un grafo di oggetti alternativo a quello usato in produzione, senza dover intervenire sul codice applicativo.

Framework e organizzazione dei test La suite è realizzata con `pytest`, affiancato dal plugin `pytest-asyncio`, configurato in modalità `auto` cosicché ogni funzione di test definita come `async def` venga eseguita direttamente sull'event loop senza bisogno di un marcatore esplicito, e da `pytest-cov` per la rilevazione della copertura. Per i test delle connessioni HTTP e WebSocket viene inoltre impiegato il `TestClient` di Starlette, basato su `httpx`, che incapsula l'applicazione ASGI dietro un'interfaccia sincrona. I test sono organizzati nella cartella `tests/api`, suddivisa in tre sotto-pacchetti – `http`, `ws` e `coap` – che rispecchiano la stessa suddivisione per protocollo già adottata nel codice applicativo descritta in sezione 5.2.1; non è invece presente una suite separata di test puramente

unitari sui singoli componenti di `/core`, poiché bus, cache, monitor e container risultano già ampiamente esercitati indirettamente da ogni test di integrazione, essendo coinvolti nel percorso completo di ciascuna richiesta. A ciascuna classe di test è inoltre associato un marcatore – `http`, `websocket` o `coap`, dichiarato in `pyproject.toml` – che consente di eseguire selettivamente la suite relativa a un solo protocollo durante lo sviluppo.

Container di test e mock della connessione MQTT Il punto di partenza di ogni test è la fixture `container`, che istanzia un nuovo `Container` e ne sostituisce l'unica connessione interna con un'implementazione fittizia, `MockMQTTConnection`. Questa classe eredita da `Connection`, rispettandone quindi il contratto, ma al momento dello `start()` si limita a sottoscrivere all'*Inbound Bus* e a ripubblicare immediatamente ogni messaggio ricevuto sull'*Outbound Bus*, senza coinvolgere alcun broker reale. Poiché la `InMemoryCache` è in ascolto proprio sull'*Outbound Bus*, questo "eco" diretto è sufficiente a riprodurre fedelmente, ai fini del test, l'intero percorso che un messaggio compie in produzione: dalla scrittura su una connessione esterna, attraverso l'*Inbound Bus*, fino alla sua ricomparsa nella cache. La semplicità di questo mock – una sola classe di poche righe – è una conseguenza diretta della dependency injection adottata in `Container`: sostituire la connessione MQTT richiede di intervenire su un'unica lista di connessioni, lasciando invariato il resto del cablaggio applicativo.

Fixture di supporto Oltre al `Container` di test, due fixture distinte espongono un client HTTP pronto all'uso: `test_client`, che imposta preventivamente lo stato della connessione MQTT simulata su `UP` prima di avviare l'applicazione, e `unhealthy_client`, che la lascia volutamente nello stato iniziale `DOWN`. Quest'ultima è impiegata specificamente nei test del controllo di salute descritto in sezione 5.2, dove un singolo test parametrizzato verifica che tutti gli endpoint coinvolti – lettura e scrittura, sia per le risorse dei robot sia per quelle di input – rispondano con 503 quando una connessione critica risulta non disponibile, e che il corpo della risposta riporti correttamente lo stato di ciascuna connessione registrata. Una seconda famiglia di fixture, raccolta in `tests/fixtures/data.py`, fornisce funzioni di fabbrica per costruire payload validi delle entità di dominio

con valori di default sovrascrivibili, evitando la duplicazione degli stessi dizionari JSON attraverso i numerosi casi di test. Per la connessione CoAP, infine, le fixture `coap_port`, `coap_server` e `coap_client` assemblano un ambiente di test indipendente da quello ASGI: la prima individua una porta UDP libera sull'interfaccia di loopback, la seconda avvia le connessioni interne del **Container** – necessario perché `MockMQTTConnection` sottoscriva il proprio eco – e costruisce un **Site** di `aiocoap` popolato con le stesse risorse usate in produzione ma collegato direttamente a cache e bus del container di test, mentre la terza apre un vero **Context** client di `aiocoap` verso quella porta, permettendo ai test di dialogare con il server tramite richieste CoAP autentiche anziché tramite un'interfaccia simulata.

Test delle connessioni esterne Per HTTP e WebSocket, il `TestClient` consente di esercitare gli endpoint con una sintassi sincrona pur attraversando l'intera applicazione reale: i test verificano, per ciascuna entità, il comportamento in lettura su cache vuota (404), la scrittura seguita da una lettura che ne conferma il valore, l'aggiornamento di valori già presenti, l'isolamento tra robot diversi e il rifiuto con 422 di payload non validi, questi ultimi spesso parametrizzati su più varianti malformate dello stesso corpo. Per WebSocket, lo stesso client apre sessioni tramite il context manager `websocket_connect`, permettendo di verificare l'intero ciclo descritto in sezione 4.2.1: creazione di una sottoscrizione tramite gli endpoint REST dedicati, associazione dell'identificativo alla sessione attiva, ricezione delle notifiche corrispondenti e relativa cessazione dopo la cancellazione della sottoscrizione, oltre alla chiusura della connessione in risposta a un frame non valido. Per CoAP, i test seguono uno schema analogo per lettura, scrittura e Observe, con un accorgimento aggiuntivo: poiché il bus interno adotta una semantica fire-and-forget un valore pubblicato non è garantito essere immediatamente visibile in cache al ritorno della chiamata di scrittura. I test che verificano l'effetto di una PUT successiva ricorrono quindi a una funzione di supporto, `wait_for_payload`, che ripete la richiesta GET a intervalli regolari fino a quando il valore atteso non compare o fino a un timeout, rendendo i test robusti rispetto alla natura asincrona della consegna.

Risultati e copertura La suite comprende complessivamente ottanta test, tutti superati in circa sei secondi di esecuzione. Il report generato da `pytest-cov` mostra una copertura complessiva dell'88% sulle istruzioni del pacchetto `domain_gateway`. La distribuzione per modulo non è tuttavia uniforme: i componenti di `/core – Connection, Container, Monitor`, e in misura quasi totale anche `Bus` e `Cache` – raggiungono una copertura prossima o pari al 100%, conseguenza diretta dell'essere attraversati da ogni singolo test di integrazione indipendentemente dal protocollo coinvolto. Il valore più basso si registra invece in `MQTTConnection`, ferma al 35%: si tratta di una conseguenza attesa della scelta di sostituirla interamente con `MockMQTTConnection` nella suite, per cui la logica di riconnessione con backoff esponenziale descritta in sezione 5.2 – che richiederebbe di simulare un broker realmente irraggiungibile per essere esercitata – non viene attualmente coperta da test automatici. Una riduzione più contenuta si osserva anche nella connessione CoAP e nel modulo `mappings.py`, riconducibile principalmente a percorsi di gestione degli errori, come il fallimento del bind di rete o un topic non riconosciuto dal dispatcher MQTT, anch'essi difficili da riprodurre in un ambiente di test senza introdurre infrastruttura aggiuntiva o complicare artificialmente la suite a fronte di un beneficio limitato.

Capitolo 7

Conclusione e Sviluppi Futuri

Bibliografia

- [ABB⁺25] Gianluca Aguzzi, Lorenzo Bacchini, Martina Baiardi, Roberto Casadei, Angela Cortecchia, Davide Domini, Nicolas Farabegoli, Danilo Pianini, and Mirko Viroli. A demonstrator for self-organizing robot teams. In Cinzia Di Giusto and António Ravara, editors, *Coordination Models and Languages - 27th IFIP WG 6.1 International Conference, COORDINATION 2025, Held as Part of the 20th International Federated Conference on Distributed Computing Techniques, DisCoTec 2025, Lille, France, June 17-19, 2025, Proceedings*, Lecture Notes in Computer Science, pages 230–244. Springer, 2025.
- [ACV24] Gianluca Aguzzi, Roberto Casadei, and Mirko Viroli. Macroswarm: A field-based compositional framework for swarm programming. *CoRR*, abs/2401.10969, 2024.
- [At] Christian Amsüss and the aiocoap contributors. aiocoap.
- [AT24] Giorgio Audrito and Gianluca Torta. FCPP to aggregate them all. *Sci. Comput. Program.*, 231:103026, 2024.
- [CJR⁺26] Samuel Colvin, Eric Jolibois, Hasan Ramezani, Adrian Garcia Badaracco, Terrence Dorsey, David Montague, Serge Matveenko, Marcelo Trylesinski, Sydney Runkle, David Hewitt, Alex Hall, and Victorien Plot. Pydantic Validation, 2026.
- [CVAP22] Roberto Casadei, Mirko Viroli, Gianluca Aguzzi, and Danilo Pianini. Scafi: A scala DSL and toolkit for aggregate programming. *SoftwareX*, 20:101248, 2022.

- [emp26] empicano. aiomqtt, 2026.
- [Har15] Klaus Hartke. Observing Resources in the Constrained Application Protocol (CoAP). RFC 7641, September 2015.
- [OEJ15] Alison Olechowski, Steven D. Eppinger, and Nitin Joglekar. Technology readiness levels at 40: A study of state-of-the-art use, challenges, and opportunities. In *2015 Portland International Conference on Management of Engineering and Technology (PICMET)*, pages 2084–2094, 2015.
- [Ope21] OpenAPI Initiative. OpenAPI Specification, February 2021.
- [Ram] Sebastián Ramírez. FastAPI.
- [TC] Marcelo Trylesinski and Tom Christie. Uvicorn.
- [uv26] uv, 2026.

Acknowledgements

Optional. Max 1 page.